2026-8-11 Linux线程--封装

- 一、 前言：为什么要封装线程?
- 二、 C++ 线程类的设计与封装
  - 2.1 线程类的核心属性
  - 2.2 核心接口设计
  - 2.3 封装过程中的关键问题与解决方案
    - 2.3.1 静态成员函数与 this 指针的冲突
    - 2.3.2 对象上下文传递
    - 2.3.3 线程命名：pthread_setname_np
- 三、 线程参数传递的演进：从模板到任务类
  - 3.1 模板化传参：支持任意类型
  - 3.2 基于任务类的解耦方案
- 四、 多线程并发安全问题 —— 以抢票为例
  - 4.1 现象：票数出现负数
  - 4.2 原因分析：临界区竞争
  - 4.3 解决方案与结论
- 五、 其他技术细节
  - 5.1 线程函数内的局部变量
  - 5.2 多线程打印的输出干扰
  - 5.3 线程内调用 exec
- 六、 总结

一、 前言：为什么要封装线程?

上节课我们重点谈论的内容叫做线程的控制，从创建线程到终止线程，再到等待线程，再到线程分离，一个线程从生到死的整个生命周期，我们已经能够对它们进行一定的控制了。
当我们把这部分内容做完了，那么后续的工作就是用执行流来完成我们想要完成的任务。
上节课我们除了线程的那些基本控制的代码之外，重点还聊了每一个线程都要有自己独立的栈结构。今天我们就知道了，线程独立的栈是在 pthread 库当中去维护的，在库当中会维护线程的描述结构体，我们叫做 TCB。那么其中也包含线程栈的大小和栈的一些信息，还包括线程所对应的返回值或者信号等信息，还有描述当前线程是否分离的相关字段。
C++ 当中对于线程的支持，本质上也是封装了我们 Linux 系统当中的原生线程。所以原生线程不封装，系统不能用。尤其是 C++11 线程库，它封装了原生线程接口，这样的话 C++ 语言就可以对 Windows、macOS 或者 Linux 各种不同平台的线程统一用同一套线程接口来做封装。
所以我们接下来要对线程进行封装，我们想以面向对象的方式来控制我们对应的线程。

二、 C++ 线程类的设计与封装

2.1 线程类的核心属性

我们先来创建一个 `Thread.hpp` 文件，对线程进行一定意义上的封装。
那么一个线程类，它应该包含哪些属性？

```
class Thread {
private:
    std::string _threadname;    // 线程名称
    pthread_t _tid;             // 线程ID
    enum Status {               // 线程状态
        NEW = 0,
        RUNNING,
        STOPPED
    } _status;
    bool _isjoinable;           // 是否可被join
};
```

线程必须有名字，除了名字之外还有 `pthread_t` 叫做线程的 TID。当然如果需要的话，我们也可以把线程所对应的 PID 也获取一下，但用 TID 就够用了。
再下来我们还需要有一个线程的状态，所以 `enum Status` 就出来了。那么线程的状态都有哪些？有新建（NEW）、运行（RUNNING），还有终止（STOPPED）。
还有一个属性：线程可以被设置为分离状态，一旦它分离了，那么当前你就不需要再去对它进行 `join` 了。所以我们设计一个布尔类型的 `_isjoinable` 表示当前线程是否可被 join。默认情况下线程必须得被 join，所以它默认为 `true`。

2.2 核心接口设计

那么线程对应的构造函数和析构函数我们也是要有的：

```
class Thread {
public:
    Thread();
    ~Thread();

    void Start();       // 启动线程
    void Join();        // 等待线程结束
    void Detach();      // 分离线程
```

```
    void Cancel();      // 取消线程
};
```

`Start` 让线程启动起来，`Join` 等待线程结束并回收资源，`Detach` 让线程分离，`Cancel` 强制终止线程。当然你也可以再设置一个获取线程名字的接口，后面更方便。

2.3 封装过程中的关键问题与解决方案

2.3.1 静态成员函数与 this 指针的冲突

当我们创建线程时，我们想在类内部定义一个静态成员函数作为线程的入口函数。但这里会出现一个问题：

```
void* Routine(void *args) {
    // 这是类的成员函数
}
```

`pthread_create` 要求的回调函数签名是 `void* (*)(void*)`，而类成员函数隐含 `this` 指针，所以它的参数其实有两个：一个是 `void*`，一个是隐含的 `this` 指针。导致函数类型不匹配，编译会报错。
解决方案：把线程入口函数声明为 `static` 修饰。

```
static void* Routine(void *args);
```

`static` 方法属于类，不属于对象，它不存在隐含的 `this` 指针。所以参数和返回值就符合 `pthread_create` 的要求了，报错自然就没了。

2.3.2 对象上下文传递

`static` 方法属于类而不属于对象，没有 `this` 指针，我们就没办法在 `Routine` 内部直接访问非静态成员变量。
解决方案：在创建线程时，把当前对象的 `this` 指针作为参数传进去，在静态入口函数中强转回对象指针。

```
void Thread::Start() {
    pthread_create(&_tid, nullptr, Routine, this);
}
```

```
void* Thread::Routine(void *args) {
    Thread* self = static_cast<Thread*>(args);
    // 现在就可以访问非静态成员了
    self->Run();
    return nullptr;
}
```

2.3.3 线程命名：pthread_setname_np

当我们想要通过 `ps` 或 `top` 命令观察线程状态时，如果能给线程起个名字会方便很多。
在 Linux 下有一个非标准函数 `pthread_setname_np`，它可以把你的轻量级进程的名字给改了。这个名字会被记录在 PCB 里。

```
pthread_setname_np(pthread_self(), _threadname.c_str());
```

这个名字的长度一般有限制，通常 16 个字符，如果超过可能会被截断。`np` 表示 "non-portable"，这个函数只能在某些 Unix 系统下跑。

> 需要注意的是，你只能设置自己的线程名字，不能设置别人的。

三、 线程参数传递的演进：从模板到任务类

3.1 模板化传参：支持任意类型

我们创建线程时，默认让它执行 `Routine` 函数。但我们的线程最终要做什么事情，完全由用户来指定。
如果我们直接在类内部写死一个无参的 `Run` 函数，那么线程就只能执行固定类型的任务，灵活性很差。所以我们需要让线程能够接受参数。
我们引入模板 `template<typename T>`，允许用户在创建线程对象时传递任意类型的参数，比如 `int`、`struct` 或 `class` 对象：

```
template<typename T>
class Thread {
    // ...
    void Start(T* arg) {
        _arg = arg;
        pthread_create(&_tid, nullptr, Routine, this);
```

```
    }
};
```

3.2 基于任务类的解耦方案

问题：直接传参可能导致线程模块与业务逻辑耦合度过高，降低了线程类的通用性。比如你的参数是 `struct Task`，那你以后每次都得传 `Task`。
优化方案：定义独立的 `Task` 类封装具体业务逻辑和数据，线程类只负责执行绑定的 `Task` 对象中的方法，保持低耦合。

```
class Task {
public:
    virtual void Run() = 0;
};

class MyTask : public Task {
public:
    void Run() override {
        // 具体的业务逻辑
    }
};
```

这样线程模块就不再关心具体参数了，参数的数量和内容完全由 `Task` 类内部决定。优势就是模块和模块之间彻底解耦，线程类保持通用性，符合"一切皆对象"的设计原则。

> **结论**：我们其实并不需要给线程传递复杂参数。把参数封装成类，比用可变参数接口要优雅得多，而且代码好维护。

四、 多线程并发安全问题 —— 以抢票为例

4.1 现象：票数出现负数

接下来我们来验证多线程并发访问共享资源的问题。我们模拟一个抢票系统：

```
int tickets = 10000;  // 全局共享变量

void* BuyTicket(void* args) {
    while (true) {
        if (tickets > 0) {
            // 模拟抢票过程
```

```
            usleep(1000);
            tickets--;
            printf("线程 %s 抢到一张票，剩余 %d\n", name, tickets);
        } else {
            break;
        }
    }
    return nullptr;
}
```

多个线程同时执行这个函数，我们期望的结果是：票数从 10000 减到 0，一张不多一张不少。
但实际运行结果中，票数出现了负数，比如 `-1`、`-2`。这就像电影院一共 100 张票，结果卖出 102 张——业务逻辑完全被破坏了。

4.2 原因分析：临界区竞争

根本原因是什么？有人说是 `tickets--` 减的问题，其实跟减关系不大，主要是 **判断** 的问题。
我们看一下当票数为 1 时会发生什么：

1. 线程 A 读取 `tickets` 的值到 CPU 寄存器，判断 `1 > 0` 为真，准备执行 `tickets--`
2. 操作系统把线程 A 切走了（时间片用完或者被抢占）
3. CPU 内的所有寄存器内容都属于线程 A 的上下文，被保存起来
4. 此时内存中的 `tickets` 还是 1，因为线程 A 还没执行减操作
5. 操作系统调度线程 B，线程 B 读取 `tickets` 也是 1，判断为真
6. 线程 B 也被切走，线程 C、D 依次进来
   这就导致了一个严重问题：`tickets` **只剩 1 的时候，只允许一个人抢，却因为并发调度，让多个线程同时进入了判断条件**。
   当这些线程依次被唤醒执行 `tickets--` 时，票数就变成了负数。
   根本原因在于：

- 判断 (`tickets > 0`) 和 递减 (`tickets--`) 是两个独立的操作，不是原子的
- 多线程并发访问临界资源时，如果没有保护机制，数据一致性就会被破坏

4.3 解决方案与结论

临界区：把多线程共享的资源叫做共享资源，被保护起来的共享资源叫临界资源，访问临界资源的代码叫临界区。

问题在于：我们对 `tickets` 判断和递减的操作没有加保护。所以我们需要保证：**当一个线程进入临界区后，其他线程不能同时进入。**
这就引出了下一个话题——**线程同步与互斥**。我们可以用互斥锁（Mutex）来保护临界区：

```
pthread_mutex_t mutex;

void* BuyTicket(void* args) {
    while (true) {
        pthread_mutex_lock(&mutex);
        if (tickets > 0) {
            tickets--;
            printf("抢到一张票，剩余 %d\n", tickets);
            pthread_mutex_unlock(&mutex);
        } else {
            pthread_mutex_unlock(&mutex);
            break;
        }
    }
    return nullptr;
}
```

结论：缺乏互斥保护机制是导致数据不一致的根本原因，需要引入锁等同步机制确保同一时刻只有一个线程访问临界区。

五、 其他技术细节

5.1 线程函数内的局部变量

每个线程都有自己独立的栈空间。当多个线程执行同一个入口函数时，函数内部定义的局部变量存储在线程自己的栈中，天然线程安全，不会互相影响。

5.2 多线程打印的输出干扰

多线程同时向显示器打印时，因为显示器也是共享资源，且一切皆文件，多个线程同时写同一个文件描述符，就会出现字符交错，信息出现干扰。

5.3 线程内调用 exec

一个线程如果调用了 `exec` 系列函数，会替换整个进程的地址空间，导致其他所有线程终止。这相当于整个进程都被替换了，需要谨慎使用。

六、 总结

今天我们学到了：

1. **线程封装**：用面向对象的方式对 POSIX 原生线程进行了封装，实现了线程的创建、启动、分离、等待和取消等生命周期管理。关键在于用 `static` 修饰入口函数，并通过传递 `this` 指针获取对象上下文。
2. **参数传递的演进**：从固定参数到模板化传参，再到基于任务类的解耦方案，最终实现了线程模块与业务逻辑的低耦合设计。
3. **并发安全问题**：通过抢票案例，我们看到了多线程并发访问共享资源会导致数据不一致的问题。根本原因在于线程切换可能发生在判断与执行之间，导致多个线程同时进入临界区。
4. **方向**：解决这个问题需要引入同步机制，比如互斥锁。这也是我们下节课要讨论的内容——**线程同步与互斥**。

> 从今天开始，你可以在多进程内部创建多个线程，每个线程又可以创建自己的子进程。多线程并发访问共享资源时，一定要考虑数据一致性问题。下节课我们正式进入线程同步的话题。